

CAS4160 Homework 5: Offline RL

[Your name]
[Your student ID]

1 Introduction

이번 과제에서는 오프라인 강화학습의 기본 알고리즘 중 하나인 Conservative Q-Learning (CQL)을 구현하게 됩니다. 제공된 다양한 하이퍼파라미터와 오프라인 데이터셋을 가지고 실험을 진행하게 될 것입니다. 이번 과제에서는 수집된 데이터의 품질에 따라 성능을 비교할 수 있도록 랜덤 정책(random policy)과 전문가 정책(expert policy)을 제공합니다.

Your objectives are as follows:

1. 오프라인 강화학습 알고리즘인 CQL을 구현하고 학습시키기
2. 주요 하이퍼파라미터를 실험하고, 이것들이 RL 에이전트의 성능에 어떻게 영향을 미치는지 탐구하기
3. Random과 Expert 등 서로 다른 오프라인 데이터셋의 품질 차이를 분석하기

Writing the report: 이 LaTeX 템플릿에서 %TODO 태그가 있는 질문들에 대한 답변을 제공하세요. 요청된 모든 값은 표로 제출하고, 요청된 모든 플롯/이미지를 삽입하세요. 또한 모든 reasoning과 텍스트 답변을 PDF에 포함시켜야 합니다.

2 Codebase

이번 과제에서는 주로 다음 코드들을 다루게 됩니다:

- cas4160/scripts/run_cql.py: CQL 구현을 위한 주요 학습 루프.
- cas4160/critics/cql_critic.py: 여러분이 구현하게 될 CQL 학습자(learner).

각 문제마다 완성해야 하는 파일들을 명시해 두었습니다. 채워야 하는 부분들은 TODO 태그로 표시되어 있습니다.

의존성 패키지들을 설치하기 위해 README.md 파일에 제공된 설정 지침을 따라주세요.

3 Tips for experiments

이번 과제에서 우리의 코드는 계산량이 가뭄습니다. RTX 3090 GPU를 사용할 경우 GPU 사용률은 최대 20% 정도입니다. GPU의 성능을 최대한 활용하기 위해, 여러 작업을 동시에 실행하는 것을 권장합니다. 이를 수행하는 방법으로는 (1) 여러 개의 bash 터미널을 열어 각 터미널에서 학습 스크립트를 실행하는 방법, (2) "&" 명령어를 사용하여 작업을 백그라운드로 실행하는 방법, 또는 (3) tmux를 사용하는 방법이 있습니다. tmux는 linux 서버에서 사용할 수 있는 강력한 유틸리티 도구 중 하나이므로 tmux를 시도해보는 것을 추천합니다. tmux의 장점 중 하나는 작업 상태가 tmux 세션 안에 저장되어, ssh 연결이 끊어지더라도 작업이 계속 실행된다는 것입니다.

4 Preliminaries

Environments: 이번 과제에서는 Pointmass 환경을 사용하게 됩니다. Pointmass 환경의 목표는 *easy, medium, hard*의 다양한 난이도를 가진 그리드 월드를 탐색하여 goal 위치에 도달하는 것입니다. 각 난이도별 샘플 환경은 Figure 1에 시각화되어 있습니다. 관찰(observation)은 point mass의 (y, x) 위치로 구성되며, 행동 공간(action space)은 NO MOVE, LEFT, RIGHT, UP, DOWN을 나타내는 5개의 이산 값입니다. 에이전트는 goal에 도달하면 보상 1을 받고 종료되며, 그렇지 않은 경우 보상 0을 받습니다.

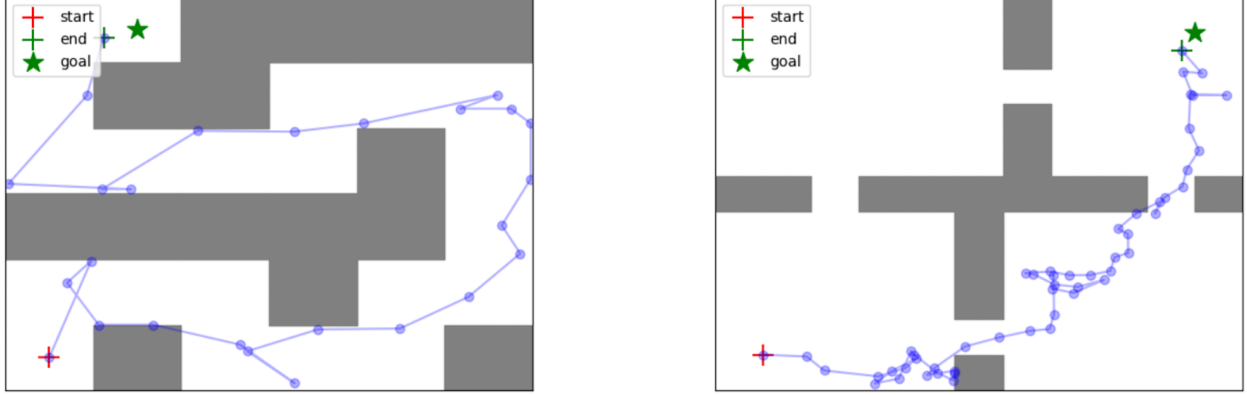


Figure 1: 내비게이션과 목표 도달을 위한 Pointmass 환경의 Medium (왼쪽)과 Hard (오른쪽) 난이도 시각화.

Offline datasets: 오프라인 RL을 위해, 우리는 오프라인 데이터셋을 수집하는 데 사용되는 데이터 수집 전략 세트를 제공했습니다. 이러한 트라젝트리들은 이후 여러분의 오프라인 RL 알고리즘의 학습 데이터셋으로 사용됩니다. 오프라인 전이 데이터셋 $\mathcal{D} = \{(s_i, a_i, r_i, s_{i+1})_i\}$ 은 현재 상태, 행동, 보상, 그리고 미래 상태의 튜플들로 구성되어 있습니다. 이번 과제에서는 (1) 랜덤 정책 (100000 steps)과 (2) 전문가 정책 (20000 steps)에 의해 생성된 두 데이터셋으로 실험하게 될 것입니다.

5 Conservative Q-Learning

5.1 Implement CQL

이번 문제에서는 **Conservative Q-Learning (CQL)** 알고리즘 (이산 환경을 위한 DQN 버전)을 구현하게 됩니다. CQL의 목표는 정책 가치의 과대평가를 방지하는 것입니다. 전체 CQL 목적 함수는 표준 TD 오차 목적 함수에 α 로 가중된 CQL 정규화 항을 추가한 형태로 주어집니다:

$$L_Q(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\left(Q(s_i, a_i) - \left(r(s_i, a_i) + \gamma \max_{a'} [Q(s'_i, a')] \right) \right)^2 \right] + \alpha \frac{1}{N} \sum_{i=1}^N \left[\log \left(\sum_{\mathbf{a}} \exp(Q(s_i, \mathbf{a})) \right) - Q(s_i, a_i) \right]. \quad (1)$$

다음 파일의 TODO를 채우세요:

- cas4160/critics/cql_critic.py

모든 TODO 명령어를 채웠다면, **expert** 데이터셋에서 다음 명령어를 사용하여 CQL을 실행할 수 있어야 합니다:

```
python cas4160/scripts/run_cql.py --env_name PointmassMedium-v0 \
  --exp_name cql_alpha_{$ALPHA}_expert \
  --dataset_name expert --cql_alpha={$ALPHA}
```

CQL을 $\alpha = \{0.0, 0.1\}$ 로 expert 데이터셋에서 실행하세요. tensorboard 로그 파일에서 마지막 스텝의 Eval_AverageReturn, Eval_StdReturn 값을 표로 보고하세요. 여러분의 CQL 구현은 Medium 난이도의 expert 데이터셋에서 $\alpha = 0.1$ 일 때 약 0.9 이상의 성능을 달성해야 합니다. 코드는 50000 iteration 동안 실행되며 약 30분 정도 소요됩니다. 또한 예 이전트의 마지막 트라젝트리를 시각화한 eval_last_traj.png 이미지와 overestimation 값 곡선, $\frac{1}{N} \sum_{i=1}^N [\log(\sum_{\mathbf{a}} \exp(Q(s_i, \mathbf{a}))) -$ 를 첨부하세요.

$\alpha = 0.0$ 은 알고리즘 측면에서 무엇을 의미하나요 (힌트: CQL이 어떤 알고리즘을 기반으로 개발되었는지 생각해 보세요)? α 에 따른 성능 차이와 overestimation 값의 차이를 설명하세요.

Answer: **Include your answer here.**

Table 1: Average Performance with different α in expert dataset. **Fill data and replace the caption.**

	Mean	Standard Deviation
$\alpha = 0.0$	-	-
$\alpha = 0.1$	-	-



Figure 2: **Provide your trajectory plot for expert dataset with $\alpha = 0.0$ and replace the caption.**

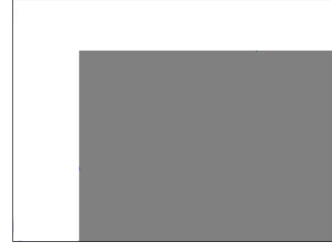


Figure 3: **Provide your trajectory plot for expert dataset with $\alpha = 0.1$ and replace the caption.**

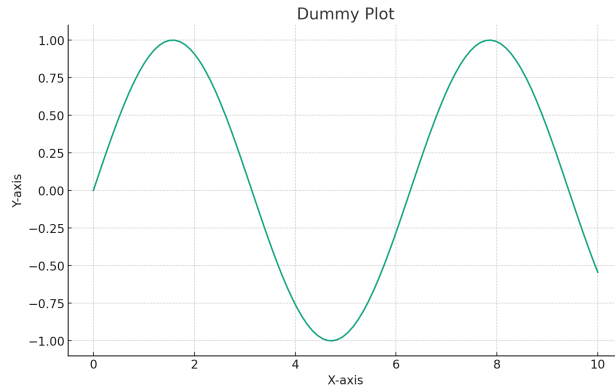


Figure 4: **Provide your plot for overestimation of Q values during the training for expert dataset with $\alpha = 0.0, 0.1$ and replace the caption.**

5.2 Comparison between Expert dataset and Random dataset (5 points)

PointmassMedium-v0 환경에서 $\alpha = \{0.0, 0.1\}$ 로 expert 데이터셋과 random 데이터셋에 대해 학습된 정책들을 비교하세요. eval average return과 standard deviation을 보고하세요. 아래 명령어를 실행하여 **random** 데이터셋으로 CQL을 실행하세요:

```
python cas4160/scripts/run_cql.py --env_name PointmassMedium-v0
--exp_name cql_alpha_{$ALPHA}_random
--dataset_name random --cql_alpha={$ALPHA}
```

각 실행(run)마다 에이전트의 마지막 트라젝트리를 나타내는 eval_last_traj.png 이미지를 첨부하세요.

random 데이터셋에서는 $\alpha = 0.0$ 과 $\alpha = 0.1$ 사이의 성능 차이가 이전 expert 데이터셋 실험에서 나타났던 것만큼 크지 않다는 것을 발견했을 수 있습니다. $\alpha = 0.0$ 이 random 데이터셋에서는 잘 수행되지만 expert 데이터셋에서는 실패하는 이유를 설명하세요. 힌트: CQL은 'unseen' 행동의 과대평가를 방지하는 것을 목표로 합니다.

Answer: **Include your answer here.**

Table 2: Average Performance with different datasets and α . **Fill data and replace the title.**

	Mean	Standard Deviation
Random; $\alpha = 0.0$	-	-
Random; $\alpha = 0.1$	-	-
Expert; $\alpha = 0.0$	-	-
Expert; $\alpha = 0.1$	-	-



Figure 5: **Provide your trajectory plot for random dataset with $\alpha = 0.0$ and replace the caption.**

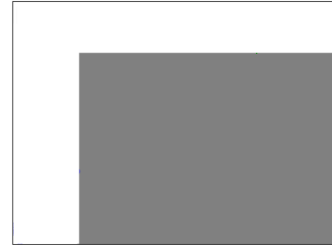


Figure 6: **Provide your trajectory plot for random dataset with $\alpha = 0.1$ and replace the caption.**

6 Submission

Please include the code and the report in a single zip file, hw5_[YourStudentID].zip. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw5_[YourStudentID].zip
├── hw5_[YourStudentID].pdf
├── cas4160
│   ├── critics
│   │   ├── cql.critic.py
│   │   └── ...
│   └── ...
└── ...
```